DELTAPROOF: Finding the Exact Rows That Differ After a Heterogeneous Database Migration

Nithin Narla
Independent Researcher
nithin.nnv@gmail.com

Suhas Jangoan
Independent Researcher
suhas.jangoan@ieee.org

Ayan Putatunda
Independent Researcher
ayanputatunda87@gmail.com

***Abstract*—When a system is re-engineered onto a different database, for example from PostgreSQL to Cassandra, its owners must show that the migrated data matches the original before they cut over. Existing tools compare row counts and aggregates, which miss value errors, or compare every row, which fails when the two stores order and encode records differently. DELTAPROOF is an open-source command-line tool and Python library that reports the exact keys that are missing, extra or different between two databases, and then shows which columns differ for each. A canonical encoder makes logically equal records hash identically across stores; each store is summarized by a sketch built from invertible Bloom lookup tables, whose comparison costs bytes proportional to the number of differences rather than the size of the tables; and parallel scans, snapshot-consistent on PostgreSQL, build the sketches. On TPC-H `lineitem` loaded into PostgreSQL and Cassandra at up to 60 million rows, DELTAPROOF found every injected fault with no false positives while exchanging 260 KB; at 0.6 and 6 million rows, a checksum of the drivers' raw values flagged every row. Tool: `github.com/nithinnnv/deltaproof`. Video: `youtube.com/watch?v=jJpkxPjW1ME`.**

***Index Terms*—data migration, reengineering, data validation, set reconciliation, NoSQL**

I. INTRODUCTION

Moving a system's data to a different database is a common reengineering step: a relational store is replaced by a wide-column store to scale writes, or a legacy schema is rebuilt in a new one. The migration is complete only when the target provably holds the same data as the source. In practice teams check row counts or column sums, which pass when a value is rounded, a timestamp is shifted or a lost row is offset by a duplicate, or they read and compare every row. Key-range checksum tools such as data-diff [1] and pt-table-checksum [2] reduce transfer but require both stores to scan in the same key order, which hash-partitioned stores such as Cassandra [3] do not provide.

Two problems make heterogeneous verification hard. First, the same logical record has different physical forms: PostgreSQL returns `CHAR(25)` values padded with spaces and Cassandra returns them trimmed; one driver returns a `date`, the other its own date type; decimals, time zones and timestamp precision differ. A checksum over raw values reports nearly every row as different. Second, the useful answer is the list of differing keys and what differs in them, which is usually tiny compared with the tables.

DELTAPROOF treats verification as set reconciliation [5]: each store becomes a set of (key, value digest) elements under a canonical encoding, and the tool computes the symmetric difference of the two sets. This paper covers verification of a quiesced snapshot, such as before cutover or after a backfill. In a migration workflow, DELTAPROOF is a gate: after the backfill and before each cutover step, a script runs `deltaproof verify`, stops on a nonzero exit status, and passes the JSON report to `deltaproof inspect` and to the backfill job that re-copies the reported rows; the same command can run in CI against staging copies whenever the schema mapping changes. It contributes:

- a command-line tool that, given a short table specification and two connection URLs, prints every missing, extra and mismatched key, and a diagnosis command that shows the differing columns (Section II);
- a canonical record encoder for relational and wide-column stores, with keys of any type (Section III);
- a sketch format, built from invertible Bloom lookup tables (IBLTs) [4], that can be produced next to each store and compared in bytes proportional to the number of differences, built by parallel scans;
- an evaluation on PostgreSQL and Cassandra with injected faults, including scan scaling and the cost of a wrong size estimate (Section IV).

II. USING DELTAPROOF

A. Describing the tables

The user writes a specification that names each table's primary key and its non-key columns with a logical type. The type, not the stores' physical types, decides when two values are equal: `d2` is an exact decimal at scale 2, `c` a fixed-width string whose padding is ignored, `t` text, `dt` a calendar date, `ts` a UTC timestamp at milliseconds, and a trailing `?` allows nulls. Key columns are integers unless typed, e.g. `key = ["region:c", "account:t"]`. Listing 1 shows the specification for TPC-H `lineitem`. One specification can list many tables; they are compared together.

Listing 1. Table specification (`lineitem.toml`)

```
[[table]]
name = "lineitem"
key = ["l_orderkey", "l_linenumber"]
columns = """l_partkey:i l_suppkey:i l_quantity:d2
  l_extendedprice:d2 l_discount:d2 l_tax:d2
  l_returnflag:c l_linestatus:c l_shipdate:dt
```

```
  l_commitdate:dt l_receiptdate:dt l_shipinstruct:c
  l_shipmode:c l_comment:t"""
```

B. Finding the differences

One command compares two databases (Listing 2). When the stores sit in different networks, `deltaproof sketch` runs next to each store and writes a sketch file, and `deltaproof diff` compares the files anywhere. A file holding every size of sketch took 20 bytes per row for 600,000 rows, little less than the rows' digests; with `--max-differences 1000` it holds only the sizes needed for up to 1,000 differences, 232 KB for 600,000 rows. If more differences turn up, `diff` falls back to an exact comparison of the stores named by `--source` and `--target`, which requires access to both stores.

Listing 2. Comparing PostgreSQL with Cassandra (`--limit 1`)

```
$ deltaproof verify --spec lineitem.toml \
    --source postgresql://user:pw@pg:5432/tpch \
    --target cassandra://cass:9042/tpch --limit 1
120 differing keys
  missing_in_target: 20
    table=lineitem l_orderkey=11655 l_linenumber=3
    ... 19 more (use --limit -1 or --json)
  extra_in_target: 40
    table=lineitem l_orderkey=18277 l_linenumber=2
    ... 39 more (use --limit -1 or --json)
  value_mismatch: 60
    table=lineitem l_orderkey=11683 l_linenumber=5
    ... 59 more (use --limit -1 or --json)
compared 260,496 bytes of sketch in 2 round trips;
rescanned 0 rows
```

C. Diagnosing and fixing them

With `--json`, the report is one object: `differences`, the estimate and sketch level used, a `cost` object (bytes, round trips, rows rescanned), and for each class (`missing_in_target`, `extra_in_target`, `value_mismatch`) a `count` and a list of keys, each with its table and key columns. `deltaproof inspect` takes that report, reads each key from both stores and prints the columns whose canonical values differ (Listing 3); differences that are only formatting, such as padding or decimal scale, are not shown, because they are not differences. Its `--json` output lists, per key, the class, whether each store has the row, and each differing column with both values. On the SF 1 run of Section IV, `inspect` named exactly one column for each of the 60 value mismatches: `l_extendedprice`, `l_shipdate` and `l_comment`, 20 each, the three injected value faults.

Listing 3. Diagnosing reported keys at SF 1 (excerpt)

```
$ deltaproof verify ... --json --limit -1 > report.json
$ deltaproof inspect --spec lineitem.toml \
    --source ... --target ... report.json --limit -1
lineitem l_orderkey=3044 l_linenumber=3: missing in target
lineitem l_orderkey=4130 l_linenumber=1: extra in target
lineitem l_orderkey=3074 l_linenumber=1: value mismatch
    l_extendedprice  source=78631.50  target=78631.51
lineitem l_orderkey=3104 l_linenumber=2: value mismatch
    l_shipdate       source=1993-12-25  target=1993-12-26
lineitem l_orderkey=4098 l_linenumber=1: value mismatch
    l_comment        source=e slyly blithely silent
                       deposits. fluff
                     target=corrupted comment
```

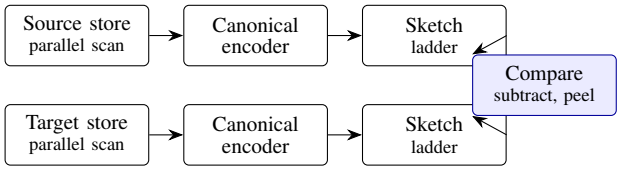


Fig. 1. Pipeline. Each side can run next to its store; only sketches travel.

TABLE I
SAME VALUE, DIFFERENT DRIVER FORMS, ONE CANONICAL FORM

Type	PostgreSQL	Cassandra	Canonical
`CHAR(10)`	`'MAIL `	`'MAIL'`	trimmed, NFC
date	`date`	`util.Date`	days since 1970
timestamp	µs	naive UTC, ms	UTC, set unit
decimal	column scale	value's scale	scaled integer

A typical repair loop exports the reported keys, re-copies those rows with the migration's own backfill or deletes the extra ones, and runs `verify` again. For automation, the exit status is 0 when the stores match, 1 when they differ and 2 when verification could not complete (for example, an unreachable store), so a cutover script can run `deltaproof verify` as a gate and attach the JSON report to the change record.

III. Architecture

Fig. 1 shows the pipeline. The library, scanners and command line are about 1,400 lines of Python with NumPy; baselines and benchmarks are separate.

Scanning. By default each store is read by up to eight worker processes, each with its own connection, reading one slice and encoding it locally; only the encoded elements, 24 bytes per row, return to the coordinator. PostgreSQL tables are sliced by physical block range, using TID range scans, so slicing does not depend on key types or skew; the coordinator exports a repeatable-read snapshot and every worker imports it, so all slices read one consistent state. Cassandra tables are sliced by token range over the table's partition key, read from cluster metadata; Cassandra has no cross-node snapshot, so the target must be quiesced during the scan. `--workers 1` uses a single cursor. A new store needs an adapter that yields rows as dictionaries, and optionally a slicing rule.

The canonical encoder turns a row into an element: a 128-bit key and a 64-bit digest of the values. Decimals become integers at the declared scale, so `10.5` and `10.50` agree; timestamps become UTC at the declared unit; text is Unicode NFC, hashed with BLAKE2b; fixed-width strings drop their padding; floats compare by their IEEE-754 bits; and nulls carry their own tag, so null, the empty string and zero differ. Table I lists differences between the two drivers' forms of the same value that the encoder absorbs; on `lineitem`, the first two alone made a raw-value checksum flag every row.

Keys. The 128-bit key carries an 8-bit table tag, so one sketch covers many tables. Keys of up to four integer columns below 2^{30} are packed directly and decode back to

their values. Any other key, such as a text or alphanumeric natural key, a key of more columns, or wide integers, is hashed with BLAKE2b into the remaining 120 bits from its canonical parts, length-prefixed so that `("E","UACME")` and `("EU","ACME")` differ. Distinct keys then collide with probability about $n^2/2^{121}$. The limit is therefore one of the packing, not of the method: a hashed key cannot be decoded from the sketch, so the tool maps reported hashes back to key values with a key-only scan of the stores.

Sketches. An IBLT is an array of cells; each element is added to three cells, and each cell keeps a count and XOR sums of keys, digests and a checksum [4]. Subtracting the target's IBLT from the source's cancels every common element, and *peeling* cells that hold a single element recovers the difference. Peeling succeeds with high probability while a table has comfortably more cells than differences (asymptotically about 1.23 cells per difference); beyond that it stalls with cells left over, which the tool detects. The difference size is unknown in advance, so each store keeps a *ladder* of IBLTs. Level i is sized for $C_i = 64 \cdot 8^i$ differences, for $i = 0, 1, \ldots$ up to the first level with $C_i \geq n/8$ for n rows (or $C_i \geq 1.25$ times `--max-differences`). It is split by key hash into $P_i = \max(1, C_i/256)$ partitions, each an IBLT of about $2C_i/P_i$ cells, two per difference. A 32-stratum estimator [5] completes the sketch. The comparison reads the estimator first, then the smallest level with 25% headroom over the estimate. A partition that fails to peel is compared row by row; if no level is large enough, the whole table is compared. An inaccurate estimate therefore costs bandwidth and time, never correctness (Section IV). In memory the ladder takes 28, 22 and 17 bytes per row at 10^5, 10^6 and 10^7 rows, falling as tables grow.

IV. Evaluation

We ran PostgreSQL 16.4 and Cassandra 4.1.6 as single nodes in Docker on one Apple M1 Pro laptop (10 cores, 16 GB), with Python 3.13. Faults were injected with SQL and CQL statements, and the ground truth was the exact difference of full scans of both stores.

Does it find the differences? On a 200,000-row synthetic ledger with 120 faults of six classes, and on TPC-H `lineitem` at scale factors 0.1, 1 and 10 with 120 faults (lost writes, decimal precision, shifted dates, corrupted text, lost deletes, phantom rows), DeltaProof reported exactly the injected keys, with no false positives (Table II). A digest of the drivers' raw values flagged 99.9% of ledger rows and every `lineitem` row at SF 0.1 and 1 (at SF 10 we did not compute it, as it needs every raw row in memory). The SF 10 sketch was sized for up to 100,000 differences (22 MB per store), and the exact difference of the two 60-million-row scans equalled both the injected set and DeltaProof's answer. TPC-H data was generated with `dbgen`; these are not TPC-H benchmark results. At SF 0.1 we ran the `deltaproof` command itself in all three ways (one command; files with every size; files limited to 1,000 differences), and each printed the same 120 keys.

TABLE II
End-to-end runs, PostgreSQL vs. Cassandra

	Ledger	TPC-H `lineitem`		
		SF 0.1	SF 1	SF 10
Rows (source)	200,000	600,552	6,001,195	59,986,032
Injected faults	120	120	120	120
Found / false pos.	120 / 0	120 / 0	120 / 0	120 / 0
Sketch bytes compared	260 KB	260 KB	260 KB	260 KB
Round trips	2	2	2	2
Raw-value digest flags	99.9%	100%	100%	—

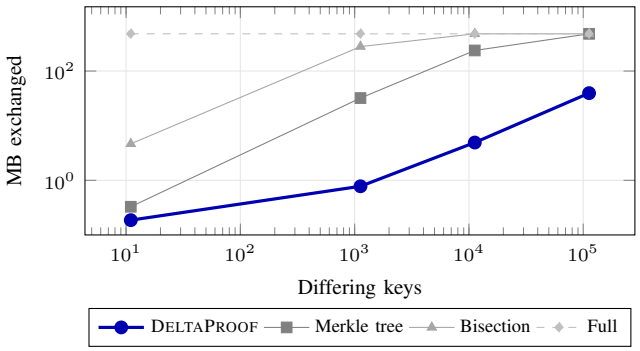


Fig. 2. Bytes exchanged to find the differences among 10^7 rows (log–log): in-process experiment on a synthetic ledger, every method under one cost model and returning the same keys.

What does a comparison cost? Comparing sketches took 12 ms for the ledger, 0.13 s for 6 million rows and 2.7 s for 60 million, always exchanging 260 KB; the time grew with rows because our harness also prepares the fallback rescan up front, hashing every key of both scanned tables, while subtracting and peeling the sketches does not depend on table size. The cost follows the number of differences (Fig. 2): in an in-process experiment with 10^7 ledger rows and about 1,100 differences, DeltaProof exchanged 0.78 MB, while a Merkle tree exchanged 32 MB, range-checksum bisection 280 MB and a full comparison 480 MB, each reporting the same keys.

How does scanning scale? Building a sketch needs one scan of each store; profiling the single-cursor scan at SF 0.1 showed that Python encoding, not fetching, took most of the time (15 s of 18 s on PostgreSQL). Each reader already streams: PostgreSQL rows arrive through server-side cursors in batches of 10,000 and Cassandra rows in pages of 5,000, so a worker's memory does not grow with the table. Parallel slices spread fetching and encoding. Table III shows scan and encode time at SF 1 (6 million rows) by worker count; every configuration produced exactly the same elements as the single cursor. Eight workers cut PostgreSQL from 176 s to 27 s and Cassandra from 293 s to 38 s (6.6× and 7.8×). At SF 10 (60 million rows), eight workers took 284 s on PostgreSQL and 424 s on Cassandra, about ten times SF 1, on a laptop that was also running both databases and swapping 7.7 GB; building both sketches took 90 s and the coordinator peaked at 4.4 GB. Our largest test is 60 million rows; for hundreds of millions of

TABLE III
SCAN AND ENCODE TIME (S), LINEITEM SF 1, BY WORKERS

Workers	1	2	4	8
PostgreSQL	176.4	92.8	49.1	26.6
Cassandra	292.5	138.6	87.4	37.6

TABLE IV
COST OF A FORCED ESTIMATE, 10^6 ROWS: MB / ROWS RESCANNED

Factor	1/16	1/4	1	4	16
1,125 keys	48.3/2.0M	0.78/0	0.78/0	4.9/0	38.0/0
11,250 keys	48.7/2.0M	5.3/15.8K	5.3/15.8K	38.1/2.0K	48.2/2.0M

rows, scan time is linear in rows per worker, so larger tables need more workers or machines; because IBLTs are linear, sketches of slices can also be built where the slices are read and summed, which would keep the coordinator's memory independent of table size (our implementation ships elements, 24 bytes per row). Computing digests inside the database would remove the transfer, but needs the canonical encoding as a user-defined function in every store.

What does a wrong estimate cost? We forced the estimate used to pick the sketch size to between $1/16$ and 16 times the strata estimate, on 10^6 ledger rows with 1,125 and 11,250 differing keys (Table IV). Every run returned exactly the differing keys. Within a factor of 4 either way, a comparison exchanged at most 38.1 MB and rescanned at most 15,791 rows. A 16-fold underestimate overloaded every partition, so nearly all 2 million rows were rescanned; a 16-fold overestimate at 11,250 differences exceeded the largest level and fell back to a full comparison. Both cost 48 MB and about half a second here, the price of a full comparison. The strata estimate itself counts elements, so it reads about 1.6 times the number of differing keys: a changed value is two elements.

V. RELATED TOOLS

Row counts and aggregates, as offered by several migration services, are cheap but miss value faults. AWS DMS data validation [6] compares rows between a migration task's source and target. Google's Data Validation Tool [7] compares counts, aggregates and row hashes across many databases, though not Cassandra. data-diff [1] and pt-table-checksum [2] checksum key ranges and narrow down mismatches, which needs both sides to scan in key order. Cassandra repairs replicas with Merkle trees [3], within one database and one encoding. DELTAPROOF differs in comparing stores with different encodings and orders, and in reporting exact keys from a transfer proportional to the number of differences. IBLTs and strata estimators come from network set reconciliation [4], [5]; PinSketch [8] sends fewer bytes at a higher computational cost, and the encoder and pipeline would work with it as well.

VI. LIMITATIONS

DELTAPROOF verifies a quiesced snapshot; a store being written during the scan yields a difference that includes the writes in flight. Because IBLTs are linear, the library also accepts inserts, updates and deletes to keep sketches current from a change stream, but this paper does not evaluate that mode. Tables without a primary key cannot be compared key by key. The table specification must match the migration's intended mapping; if both sides were encoded wrongly in the same way, a mismatch would be hidden, so specifications need their own tests. Adapters exist for PostgreSQL and Cassandra only. A run with no differences still exchanges the 187 KB estimator.

VII. CONCLUSION AND AVAILABILITY

DELTAPROOF turns migration verification into a short specification and two commands: one returns the exact differing keys from stores that encode and order data differently, and the other shows what differs in each. The tool, tests, benchmark scripts, Docker setup and all result files are available under the Apache 2.0 license at `github.com/nithinnnv/deltaproof`, and a demonstration video (under 4 minutes) is at `youtube.com/watch?v=jJpkxPjW1ME`.

ACKNOWLEDGMENT

The authors used Claude (Anthropic) to write the tool's implementation, tests and benchmark scripts and to draft and edit the text of all sections of this paper, including the figure and tables. All results come from running that code. The authors reviewed and take responsibility for all content.

REFERENCES

[1] data-diff, "Technical explanation," project documentation. [Online]. Available: https://data-diff.readthedocs.io/en/latest/technical-explanation.html

[2] Percona, "pt-table-checksum," Percona Toolkit documentation. [Online]. Available: https://docs.percona.com/percona-toolkit/pt-table-checksum.html

[3] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *ACM SIGOPS Oper. Syst. Rev.*, vol. 44, no. 2, pp. 35–40, 2010.

[4] M. T. Goodrich and M. Mitzenmacher, "Invertible Bloom lookup tables," in *Proc. 49th Annu. Allerton Conf. Commun., Control, Comput.*, 2011, pp. 792–799.

[5] D. Eppstein, M. T. Goodrich, F. Uyeda, and G. Varghese, "What's the difference? Efficient set reconciliation without prior context," in *Proc. ACM SIGCOMM*, 2011, pp. 218–229.

[6] Amazon Web Services, "AWS DMS data validation," AWS Database Migration Service User Guide. [Online]. Available: https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Validating.html

[7] Google Cloud, "Data Validation Tool," GitHub repository. [Online]. Available: https://github.com/GoogleCloudPlatform/professional-services-data-validator

[8] P. Wuille, G. Maxwell, and G. Naumenko, "minisketch: An optimized library for BCH-based set reconciliation," GitHub repository. [Online]. Available: https://github.com/bitcoin-core/minisketch